미술관 도슨트 (아트로그) — 팀 역할 분담 기획서 v2

Oct 12, 2026 · @고태민

0. v1(10/8) 대비 바뀐 점

10/8 기획서의 RAG 목표와 역할 층(A→E)은 그대로 두고, 실제로 구현 중인 기술 스택과 데이터 현황에 맞춰 고쳤다. 화면은 Streamlit 대신 Flutter Web, 사용자 기록은 SQLite 대신 TiDB Cloud로 바뀌었고 벡터 DB는 그대로 Chroma를 쓰며, 배포는 Vercel(프론트)과 Railway(백엔드)로 한다.

항목	v1 기획 (10/8)	v2 현재
화면	Streamlit	Flutter Web (서비스명 가칭 "아트로그")
백엔드	Streamlit 안에서 체인 직접 호출	FastAPI REST API (`/api/...`). 로그인은 Flutter의 Firebase Auth, 서버는 ID 토큰만 검증
벡터 DB	Chroma	Chroma 유지(검색용). Railway에 별도 서비스로 띄우고 볼륨에 저장
작품·문서·사용자 기록 DB	SQLite (사용자 기록만)	TiDB Cloud Starter (카탈로그·문서 원문·출처·청크·벡터 원본·좋아요·팔로우·열람·채팅 기록)
채팅 기록	저장 안 함 (세션 상태)	로그인 사용자는 TiDB `chat_sessions` · `chat_messages`에 저장. 작품당 대화 1개, 다시 열면 이어짐
임베딩	OpenAI 임베딩 (미확정)	`text-embedding-3-small` 1536차원으로 고정
검색 방식	Chroma 메타데이터 필터	Chroma `where` 필터로 고른 작품 범위만 검색하고, 작품과 연결된 연표 사건은 TiDB에서 항상 추가
문서 스키마	JSON 6필드 (artist, title, year, doc_type, source_url, content)	`doc_key`·출처·라이선스·관련 작품·근거 문서까지 포함한 확장 스키마, TiDB 테이블 15개
작가 범위	10명	8명 수집 완료, 클림트·실레는 미수집 (거절 테스트에 활용)
배포	정하지 않음	Vercel (Flutter Web 정적 빌드) + Railway (FastAPI·Chroma) + TiDB Cloud
Git 브랜치	`a` ~ `e`	담당자 이름 브랜치 (`taemin`, `YOONJUNG`, `lsh`, `soohwi`) → `dev` → `main`

1. 프로젝트 개요

수집한 명화 작품·생애 문서만 근거로 답하는 미술관 도슨트 웹 서비스다. 밀리의서재처럼 작품을 고르면 해설을 읽고, 궁금한 점은 그 작품 범위 안에서 챗봇에게 묻는다.

- **주제(과제 요구)**: LLM을 연동한 내외부 문서 기반 질의응답 시스템
- **목표**: 문서를 청크·임베딩해 Chroma에 저장하고, LangChain으로 LLM과 연동해 문서 안에서만 답하는 RAG 챗봇
- **차별점**: 모든 답변에 근거 출처(문서 제목·출처명·URL·라이선스)를 표시하고, 범위 밖 질문(피카소, 미수집 작가인 클림트·실레)은 "자료에 없다"고 답해 환각 방지를 보여 준다
- **데모 포인트**: 작가 간 비교 질문("고흐와 고갱은 어떤 관계였어?", "렘브란트와 카라바조의 명암 차이는?")과 판본 구분 질문(씨 뿌리는 사람·랑글루아 다리)

구분	작가	상태
르네상스·바로크	렘브란트, 카라바조	수집 완료
인상주의·후기인상주의	클로드 모네, 폴 세잔, 빈센트 반 고흐, 폴 고갱	수집 완료
조선 회화	김홍도, 정선	수집 완료
빈 분리파·표현주의	구스타프 클림트, 에곤 실레	미수집. 발표까지 채우지 못하면 "다루지 않는 작가" 거절 예시로 사용

1-1. 서비스 기능

작품을 고르고 → 대화로 깊게 듣고 → 비슷한 작품으로 이어지고 → 내 서재에 기록이 쌓이는 흐름이다. 평가의 중심은 RAG 질의응답이므로 P0부터 완성한다. Flutter 화면은 미리보기 모드로 전 기능이 만들어져 있고, 남은 일은 FastAPI 엔드포인트와 실제 연결이다.

우선순위	기능	동작	API	담당
P0	작품 목록·상세	사조·작가 필터, 검색, 작품 상세와 이미지 출처	`GET /api/artworks`, `/api/artists`, `/api/movements`	A(데이터), D
P0	작품 도슨트 챗봇	고른 작품 범위 문서만 검색해 답변, 멀티턴 유지(비로그인은 직전 12개 메시지를 함께 전송), 범위 밖이면 `refused: true`	`POST /api/artworks/{id}/chat`	B, C, D

우선순위	기능	동작	API	담당
P0	답변 출처 표시	근거 문서 제목·출처명·URL·라이선스, 보충·보정문의 원근거	chat 응답 `sources`	C, D
P1	로그인	Flutter에서 Firebase Auth로 로그인·가입. 요청마다 ID 토큰을 보내고 서버가 검증해 uid로 사용자를 찾음 (처음이면 `users`에 생성)	로그인 API 없음. 모든 인증 요청에 `Authorization: Bearer <ID 토큰>`	E, D
P1	좋아요·비슷한 작품 추천	좋아요 저장, 작품 벡터 기준 유사 작품	`PUT/DELETE .../like`, `GET .../similar`	B, E, D
P2	채팅 기록	로그인 사용자의 질문·답변·출처 저장, 작품을 다시 열면 대화 복원, 내 대화 목록, 질문 기반 추천 신호	`GET /api/artworks/{id}/chat/messages`, `DELETE /api/artworks/{id}/chat`, `GET /api/me/chats`	C, E, D
P2	작가 팔로우	팔로우한 작가 작품을 목록 상단에 노출	`PUT/DELETE .../follow`	E, D
P2	내 서재 캘린더	서울 기준 날짜별 열람 기록, 월별 달력	`POST .../views`, `GET /api/me/library/...`	E, D

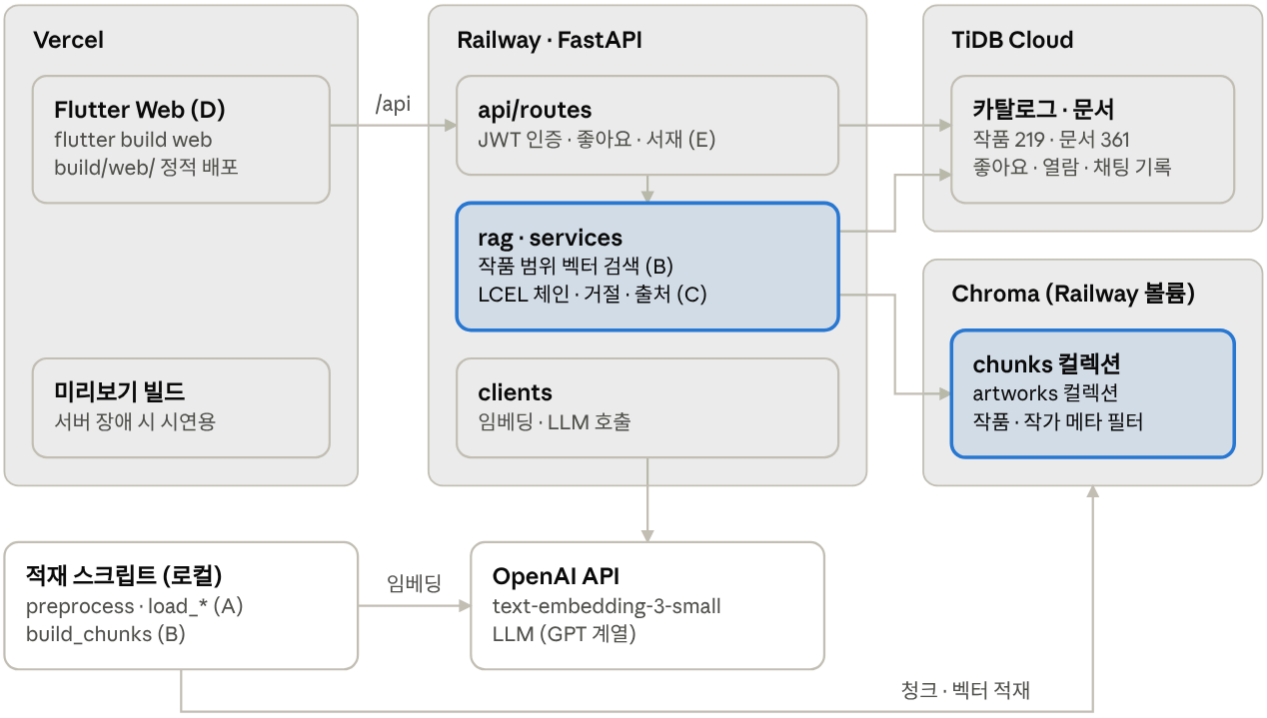
P0는 10/13 통합 v1까지 필수, P1은 10/14 오전, P2는 시간이 남을 때 붙인다. 사용자 기록은 v1의 SQLite 대신 TiDB에 저장한다. Railway 컨테이너는 재배포 때 로컬 파일이 사라지므로 SQLite 파일을 쓸 수 없다.

2. 시스템 아키텍처

Flutter Web(Vercel)이 FastAPI(Railway)를 부르고, FastAPI가 Chroma에서 작품 범위 문서를 벡터 검색하고 TiDB에서 출처를 붙여 LLM에 근거로 넘긴다. 문서 적재와 임베딩은 배포 서버가 아니라 로컬에서 스크립트로 한 번 돌린다.

요청은 Vercel → Railway → TiDB·Chroma, 적재는 로컬 스크립트

괄호 안은 담당 역할 · 강조 색은 RAG 핵심 단계



서비스 구성 · 배포 3곳 + 로컬 적재

데이터가 답변이 되기까지의 순서가 곧 역할 A→B→C→D 순서다.

1. **수집(전원)**: 작가 생애·작품 해설 문서를 `data/raw/작가명/` 에 저장
2. **전처리(A)**: `scripts/preprocess.py` 로 정제·메타데이터 부착 → `data/processed/*.jsonl`
3. **적재(A)**: `load_catalog` , `load_documents` 로 TiDB에 사조·작가·작품·문서 저장 (완료)
4. **청킹·임베딩(B)**: TiDB에서 문서를 읽어 종류별로 자르고 `text-embedding-3-small` 로 벡터화 → TiDB `document_chunks` · `artwork_embeddings` 에 저장하고 `sync_chroma` 로 Chroma 청크 컬렉션·작품 컬렉션에 옮김. Chroma 메타데이터에 `document_id` , `doc_key` , `doc_type` , `artwork_id` 를 넣는다
5. **검색(B)**: Chroma `where` 필터로 고른 작품의 해설·작가 생애·비교 문서만 남겨 상위 k개. 작품과 연결된 연표 사건(`document_artworks`)은 거리와 상관없이 TiDB에서 가져와 포함
6. **생성(C)**: LangChain LCEL 체인이 검색 문단 + 질문 + 대화 이력으로 LLM 호출, 근거가 없으면 `refused: true` . 출처는 `document_id` 로 TiDB에서 조회
7. **화면(D)**: Flutter가 `POST /api/artworks/{id}/chat` 을 호출해 답변과 출처를 표시

기술 스택

구성 요소	선택	비고
언어·패키지	Python 3.12, uv	`pyproject.toml` · `uv.lock` 은 저장소 루트
백엔드	FastAPI, SQLAlchemy, Alembic, PyMySQL	마이그레이션 `20261012_0003` 까지 적용
LLM 연동	LangChain (LCEL), `langchain-openai`, `langchain-chroma`	`chromadb` · `langchain-chroma` 는 아직 의존성에 없음
LLM	OpenAI GPT 계열	답변용 모델명은 C가 확정해 `docs/prompts.md` 에 기록
임베딩	`text-embedding-3-small` (1536차원)	바꾸면 Chroma 컬렉션 전체 재적재 필요
벡터 DB	Chroma	청크·작품 벡터, 메타데이터 필터로 작품 범위 검색
DB	TiDB Cloud Starter	카탈로그·문서 원문·출처·사용자 기록. 청크·벡터 원본(`document_chunks`, `artwork_embeddings`)도 저장해 Chroma 복원·검색 비교에 씀
인증	Firebase Auth (Flutter `firebase_auth`), 서버 토큰 검증 `firebase-admin`	uid는 검증한 ID 토큰에서만 꺼냄. 비밀번호는 저장하지 않음
화면	Flutter Web (Dart 3.10 이상)	해시 라우팅, 미리보기/실제 API 모드
배포	Vercel, Railway	아래 표
협업	GitHub, Discord	—

배포 구성

대상	플랫폼	빌드·실행	환경변수
프론트	Vercel	`flutter build web --release --dart-define=ARTLOG_DEMO=false --dart-define=API_BASE_URL=<Railway 주소>` 후 `build/web/` 을 정적 배포. 해시 라우팅이라 rewrite 설정 불필요	없음. 값은 빌드 때 고정되고 비밀값은 넣지 않음
백엔드	Railway	루트에서 `uv sync` 후 `backend/` 에서 `uv run uvicorn app.main:app --host 0.0.0.0 --port $PORT`	`TIDB_*`, `OPENAI_API_KEY`, `CHROMA_*`, Firebase 서비스 계정 키, `CORS_ORIGINS` (Vercel 도메인), `APP_TIMEZONE=Asia/Seoul`
벡터 DB	Railway (별도 서비스)	Chroma 공식 Docker 이미지 + 볼륨. FastAPI와 적재 스크립트는 `HttpClient` 로 접속	접속 주소·포트, 인증 토큰
DB	TiDB Cloud Starter	이미 운영 중, TLS 접속	개발용·발표용 데이터베이스 이름 분리

적재·임베딩 스크립트는 Railway에서 돌리지 않는다. 로컬에서 발표용 TiDB·Chroma를 대상으로 한 번 실행한다.

3. 역할 분담

층 구조(데이터 → 검색 → 생성 → 화면 → 테스트·사용자 기록)는 v1과 같다. 바뀐 것은 도구와, 화면과 백엔드 사이에 FastAPI가 들어가 API 명세가 세 번째 약속이 된 점이다.

역할	핵심 업무	책임 산출물
A. PM·데이터·DB	문서 스키마·수집 기준, 전처리, TiDB 스키마·Alembic, 카탈로그·문서 적재, 일정 관리, PPT 취합	수집·전처리 문서, DB 설계 문서, 적재 스크립트, 발표 PPT
B. 검색 (Retrieval)	Chroma 컬렉션 설계, 청크·임베딩 스크립트 (`build_chunks.py`, `build_artwork_embeddings.py`), 작품 범위 벡터 검색(`app/rag`), 유사 작품 추천	시스템 아키텍처 문서, 검색·추천 코드

역할	핵심 업무	책임 산출물
C. 생성(LLM)	LangChain LCEL 체인, 멀티턴, 시스템 프롬프트 8개 규칙(범위·판본 구분·보정문 우선 등), 거절 판단, 출처 조립, `POST /chat` 엔드포인트	RAG 체인 코드, `docs/prompts.md` 프롬프트 버전 기록
D. 화면 (Flutter)	Flutter Web 화면(발견하기·작품 상세·도슨트·작가·내 서재), 미리보기 → 실제 API 모드 전환, Firebase 로그인, Vercel 배포	시연 화면, `frontend/` 코드, 엔드포인트 점검표
E. 테스트·사용자 기록	Firebase 토큰 검증·좋아요·팔로우·열람·채팅 기록 FastAPI 라우터, 팔로우 우선 노출, RAG 평가 질문 30개 실행·집계, 데모 시나리오, Railway 배포	테스트 계획·결과 보고서, 사용자 기록 API 코드

먼저 고정할 약속 3가지:

1. **문서 스키마 (A, 확정)**: `data/processed/documents.jsonl`과 `documents` 테이블. 필드 설명은 `docs/rag-handoff.md` 2절
2. **검색 결과 형식 (B)**: LangChain `Document(page_content, metadata)` 리스트. metadata에 `document_id`, `doc_key`, `title`, `source_name`, `source_url`, `license`, `distance`를 넣는다. C는 이 형식만 받으면 가짜 데이터로 체인을 먼저 만들 수 있다
3. **API 요청·응답 형식 (C·D·E)**: Flutter가 쓰는 경로는 `frontend/docs/endpoints.md`에 있다. 로그인 형식, 목록 페이지네이션, 캘린더 응답은 아직 임시라서 10/13 오전까지 `docs/api.md`로 확정한다

발표는 15분, 1인 약 3분씩 각자 맡은 층을 설명한다.

4. 데이터 수집 현황

작가 8명, 작품 219점, 문서 361건이 전처리·검토를 마치고 TiDB에 적재됐다(10/12 기준). 작가당 최소 기준(생애 1개 + 대표작 5점)은 8명 모두 넘겼다. 남은 일은 이 문서를 청크·임베딩해 TiDB에 저장하고 검색용으로 Chroma에 옮기는 것이며 B가 맡는다.

담당	작가	작품	작품 해설 문서	생애·연표·주변 인물 문서	비교 문서
A	김홍도, 정선	60	69	20	풍속화·진경산수 비교는 미작성
B	빈센트 반 고흐, 폴 고갱	50	50	34	1 (아를 공동생활)
C	렘브란트, 카라바조	54	58	40	4 (테네브리즘·키아로스쿠로 원문·번역)
D	구스타프 클림트, 에곤 실레	0	0	0	미수집

담당	작가	작품	작품 해설 문서	생애·연표·주변 인물 문서	비교 문서
E	클로드 모네, 폴 세잔	55	55	30	인상주의·후기인상주의 비교는 미작성
합계	8명	219	232	124	5

- **출처 규칙**: 모든 문서에 `source_url` · `source_name` · `license` 를 남긴다. 위키백과는 CC BY-SA, 국립중앙박물관 큐레이터 해설은 공공누리 제3·4유형이라 답변에서 원문을 길게 옮기지 않는다.
- **번역본**: 영문 위키 59건은 한국어로 기계 번역했고, 원문 대조·수정 후 승인했다.
- **보충·보정문**: `## 감상 포인트` (팀 보충문)와 `## 답변 시 주의` (오류 보정)는 출처를 따로 연결했다.
- **저장 위치**: 원본 `data/raw/작가명/` , 전처리 결과 `data/processed/` , 자세한 기준은 `docs/data-collection.md` 와 `docs/data-spec.pdf` .

D 담당 수집이 비어 있어, 10/13 오전까지 클림트·실레를 넣을지 거절 예시로 남길지 정해야 한다. 넣으려면 작가당 생애 1 + 대표작 3점이 최소이고, A의 검토와 적재까지 반나절이 더 걸린다.

5. 남은 일정

핵심 마감은 10/13(화) 배포된 주소에서 P0 통합 v1을 한 번 돌리는 것이다. 로컬에서만 도는 상태로 10/14까지 가면 CORS·환경변수 문제를 발표 직전에 만나므로, 빈 라우터라도 10/12에 먼저 배포해 둔다. 10/8~10/11 작업(스키마 합의, 수집, 전처리, DB 적재, Flutter 화면)은 끝났다.

날짜	전원	A 데이터·DB	B 검색	C 생성	D 화면	E 테스트·기록
10/12(월)	API 명세 초안 합의	전처리·적재 완료, FastAPI `main.py` 뼈대와 카탈로그 조회 API	청크·임베딩 Chroma 전체 적재	프롬프트·거절 규칙, 가짜 문서로 체인	Vercel에 미리보기 모드 배포	Railway에 백엔드 뼈대 배포, 질문셋 확정
10/13(화)	배포 주소에서 통합 v1 실행	`docs/api.md` 확정, 데이터 보완	범위 검색 함수, 아키텍처 문서	`/chat` 엔드포인트, 출처 표시	실제 API 모드로 전환, 도슨트 연결	Firebase 토큰 검증·좋아요 API, 1차 RAG 평가
10/14(수)	튜닝·버그 수정	PPT 취합	검색 튜닝, 유사 작품 추천	답변 품질 개선	추천·서재 화면 연결, 리허설 준비	팔로우·열람·채팅 기록 API, 결과 보고서, 데모 시나리오
10/15(목)	오전 리허설(15분), 15:00 발표	시연용 DB 점검	—	—	배포본 최종 확인	—

발표용 시연 데이터(데모 계정의 좋아요·열람 기록)는 `scripts/seed_demo_users.py` 로 넣는다. 담당자는 아직 정해지지 않았다.

6. 필수 산출물 체크리스트

필수 산출물 5종 중 데이터 문서는 끝났고, RAG 코드와 테스트 결과가 가장 많이 남아 있다.

- [x] ~~수집된 데이터 및 전처리 문서 — A (`docs/data-collection.md`, `docs/data-spec.pdf`, `docs/rag-handoff.md`, `docs/db/db-design.md`)~~
- [ ] 시스템 아키텍처 — B (`docs/architecture.md`, 2장 그림과 배포 구성 포함)
- [ ] RAG 기반 LLM과 벡터 DB 연동 코드 (GitHub) — B, C, D, E (청크·임베딩, 검색, 체인, FastAPI 라우터, Flutter 연결)
- [ ] 테스트 계획 및 결과 보고서 — E (계획 `docs/test-plan.md` 완료, 결과 `docs/test-result.md` 미작성)
- [ ] 발표 PPT — A 취합, 각자 맡은 층 슬라이드

테스트 질문셋(E)은 30문항으로 확정했다: 사실 확인 10, 세부 관찰·해석 8, 비교 4, 범위 밖 3, 근거 부족 3, 멀티턴 2. 목표는 검색 근거 적중률 90% 이상, 범위 밖·근거 부족 거절률 100%, 출처 포함률 100%이다. `docs/rag-handoff.md` 9절의 검색 테스트 27개와 합쳐 쓴다.

7. 리스크와 협업 규칙

수집 지연 리스크는 지났다. 이제 가장 큰 리스크는 FastAPI 라우터가 아직 없어 화면·검색·생성이 실제로 맞물린 적이 없다는 점과, 배포 환경 문제다.

리스크	대응
통합 시점에 API 형식 불일치	10/13 오전 `docs/api.md` 확정. Flutter는 `lib/api/artlog_api.dart`와 `lib/models.dart`만 고친다
기능 범위 과다로 RAG 미완성	P0 → P1 → P2 순서 고정. P2가 10/14 오후까지 안 되면 Flutter 미리보기 모드 화면으로만 소개
CORS·API 주소 오류	Railway `CORS_ORIGINS`에 Vercel 배포 도메인을 넣는다. `API_BASE_URL`은 Flutter 빌드 때 고정되므로 Railway 주소가 바뀌면 다시 빌드한다
Vercel에서 Flutter 빌드 실패	Vercel 빌드 환경에는 Flutter SDK가 없다. 로컬이나 GitHub Actions에서 `flutter build web`을 돌리고 `build/web/`만 올리는 방식을 기본으로 한다
클라이언트가 보낸 uid를 그대로 믿음	uid를 본문·경로로 받지 않는다. 서버는 `firebase-admin`으로 ID 토큰을 검증한 뒤 그 안의 uid만 쓴다. E가 잘못된·만료 토큰 401 테스트로 확인

리스크	대응
Firebase 로그인 도메인 거부	Firebase 콘솔의 승인된 도메인에 Vercel 배포 도메인을 추가한다. 발표용 데모 계정은 Firebase에서 만들고 그 uid로 시연 데이터를 넣는다
Chroma 벡터 유실	Railway 볼륨을 붙이지 않으면 재배포 때 벡터가 사라진다. 볼륨을 연결하고, 사라져도 TiDB에 있는 벡터를 `sync_chroma`로 다시 옮기면 복원된다(임베딩 재호출 없음)
TiDB와 Chroma 내용 불일치	TiDB를 원본으로 두고 Chroma id를 TiDB 행 id로 맞춘다. `sync_chroma`가 id 기준으로 추가·삭제하고, 적재 후 `build_chunks` → `sync_chroma` 순서로 돌린다
TiDB 연결 끊김	TiDB Cloud Starter는 오래 열린 연결을 끊는다. `pool_recycle`과 `pool_pre_ping`을 유지한다
개발·발표 데이터 섞임	TiDB 데이터베이스 이름과 Chroma 컬렉션 이름을 개발용·발표용으로 나눈다. Railway는 발표용을 본다
범위 밖 질문에 그럴듯하게 답함	프롬프트 거절 규칙 + E의 범위 밖·근거 부족 질문 6개로 확인
API 키·DB 비밀번호·Firebase 서비스 계정 키 유출	로컬은 루트 `.env`, 배포는 Railway 환경변수에만 둔다. Flutter 코드와 Vercel에는 비밀값을 넣지 않는다 (Firebase 웹 설정값은 공개돼도 되는 값)
발표 중 서버 장애	Flutter 미리보기 모드 빌드를 별도 주소로 하나 더 띄워 둔다
발표 시간 초과	10/15 오전 리허설에서 15분 측정, 1인 3분

Git 규칙

- 브랜치는 담당자 이름(`taemin`, `YOONJUNG`, `lsh`, `soohwi`) → `dev` → `main` 3단계
- 각자 자기 브랜치에서 작업 후 `dev`로 PR, 다른 1명 확인 후 merge
- `main`에는 직접 push하지 않는다. `dev`에서 전체 실행이 확인된 버전만 merge(10/13 통합 v1, 10/15 발표 직전 최종본)
- 배포는 `main` 기준으로 한다. Railway와 Vercel의 자동 배포를 `main`에만 연결하고, 통합 테스트는 `dev` 미리보기 배포에서 한다
- 작업 전 항상 `dev`를 자기 브랜치로 받아와 충돌을 작게 유지
- `data/processed/`와 DB의 `documents`는 직접 고치지 않는다. 원본을 고친 뒤 전처리·적재를 다시 돌린다

열린 질문: 원격 담당자 브랜치는 4개인데 역할은 A~E 5개다. 역할별 담당자 이름을 채워야 한다.